

Les fonctions

Une fonction est un bloc de code qui ne s'exécute que lorsqu'elle est appelée.

On peut transmettre des données, appelées paramètres, à une fonction et elle peut renvoyer des résultats en conséquence.

On peut faire un parallèle avec les fonctions en mathématiques :

Notation en mathématiques pour la fonction f :

$$f:x \mapsto 3x+1$$

Calcul de l'image de 5 :

$$f(5) = 16$$

Notation en python :

```
def f(x):  
    return (3*x+1)
```

Appel de la fonction pour calculer l'image de 5 :

```
f(5)
```

Déclaration d'une fonction

La déclaration d'une fonction se fait avec le mot-clé `def`

Voici la forme générale :

```
def ma_fonction([arguments]):  
    '''  
    in: présentation des arguments  
    out: présentation des variables renvoyées  
    '''  
    ...  
    instructions  
    ...  
    [return résultat]
```

Les crochets signalent les éléments facultatifs.

On déclare les arguments, s'il y en a, séparés par des virgules.

La docstring entre les 3 guillemets est accessible dans la console en tapant `help([nom de fonction])`

Toutes les instructions sont indentées (espacées de 3 espaces par rapport au bord)

On indique si une valeur est retournée avec le mot-clé `return`

Attention : L'exécution du `return` fait sortir du bloc d'instruction de la fonction.

Remarque : Si la fonction ne renvoie rien, on parle de procédure

Déclaration des types des variables

Même si ce n'est pas obligatoire en python, contrairement à d'autres langages comme le C, il est possible de déclarer le type des variables utilisées de la façon suivante :

```
def en_cours(heure:int) -> bool:
    """in: nb entier désignant l'heure
    out: booleen indiquant si le lycée est ouvert
    """
    lycee_ouvert = False
    if heure >= 8:
        if heure < 19:
            lycee_ouvert = True
    return lycee_ouvert
```

Activité :

Dans la fonction ci-dessus, précisez les paramètres, le retour de la fonction, leurs types.

Que fait l'instruction suivante :

```
exemple = en_cours(5)
```

Appel d'une fonction

On peut appeler une fonction :

- avec des arguments positionnels : ils sont ordonnés comme dans une liste et obligatoire :

```
def ma_fonction(arg1,arg2):
    print (arg1,'et',arg2)
```

`ma_fonction(5,3)` renvoie : 5 et 3

- avec des mots clef : ils possèdent une clé et ne sont pas ordonnés

```
def ma_fonction(arg1=6,arg2=4):
    print (arg1,'et',arg2)
```

`ma_fonction(arg1=5,arg2=3)` renvoie : 5 et 3

`ma_fonction(5,3)` renvoie : 5 et 3

`ma_fonction(arg2=3,arg1=5)` renvoie : 5 et 3

`ma_fonction()` renvoie : 6 et 4

On peut écrire des arguments positionnels et des arguments avec mot clé :

```
def ma_fonction2(arg1,arg2,arg3=4):
    print (arg1,'et',arg2, 'et',arg3)
```

Tester les instructions suivantes et analyser les erreurs : `ma_fonction2(5,arg3=2)` ;

`ma_fonction2(arg3=2,5,3)`

Variables locales et globales

Les variables utilisées et modifiées dans les fonctions sont locales, elles sont effacées après l'exécution de la fonction(ramasse-miettes) :

```
z=8
x=5
def change1(z):
    x=3
    return(z)
```

```
print(change1(7), 'et ', x)
```

Affiche dans la console :

Si on veut qu'une fonction modifie une variable qui sera utilisée à l'extérieur de la fonction, alors il faut déclarer cette variable en global :

```
z=8
x=5
def change2(z):
    global x
    x=3
    return(z)
```

```
print(change2(7), 'et ', x)
```

Affiche dans la console:

Tester les instructions ci-dessus et analyser les affichages dans la console.

Fonction lambda

Lorsque le bloc d'une fonction est réduit à une instruction on peut utiliser la notation fonction **lambda** , la fonction lambda prend un seul paramètre et renvoie une variable.

```
def fonction_f(x):  
    return 2*x+1
```

peut s'écrire

```
fonction_f = lambda x: 2*x+1
```

Activité :

En utilisant l'exemple, écrire dans un éditeur python la fonction lambda « plus_10_pour_cent » qui renvoie la valeur d'un nombre x augmenté de 10 %.

Tester sa fonction lambda dans la console en écrivant : `plus_10_pour_cent(200)`

On doit obtenir : 220

Web : <https://info.blaisepascal.fr/python-fonctions-et-procedures>